

Pre-trained Language Models for the Legal Domain: A Case Study on Indian Law

Shounak Paul

Indian Institute of Technology Kharagpur
Kharagpur, West Bengal, India

Pawan Goyal

Indian Institute of Technology Kharagpur
Kharagpur, West Bengal, India

Arpan Mandal

Indian Institute of Technology Kharagpur
Kharagpur, West Bengal, India

Saptarshi Ghosh

Indian Institute of Technology Kharagpur
Kharagpur, West Bengal, India

ABSTRACT

NLP in the legal domain has seen increasing success with the emergence of Transformer-based Pre-trained Language Models (PLMs) pre-trained on legal text. PLMs trained over European and US legal text are available publicly; however, legal text from other domains (countries), such as India, have a lot of distinguishing characteristics. With the rapidly increasing volume of Legal NLP applications in various countries, it has become necessary to pre-train such LMs over legal text of other countries as well. In this work, we attempt to investigate pre-training in the Indian legal domain. We re-train (continue pre-training) two popular legal PLMs, LegalBERT and CaseLawBERT, on Indian legal data, as well as train a model from scratch with a vocabulary based on Indian legal text. We apply these PLMs over three benchmark legal NLP tasks – Legal Statute Identification from facts, Semantic Segmentation of Court Judgment Documents, and Court Appeal Judgment Prediction – over both Indian and non-Indian (EU, UK) datasets. We observe that our approach not only enhances performance on the new domain (Indian texts) but also over the original domain (European and UK texts). We also conduct explainability experiments for a qualitative comparison of all these different PLMs.

CCS CONCEPTS

• **Computing methodologies** → Natural language processing; *Neural networks*; • **Applied computing** → Law, social and behavioral sciences.

KEYWORDS

pre-trained language models, legal domain pre-training

1 INTRODUCTION

The introduction of Transformer-based Pre-trained Language Models (PLMs) and architectures such as BERT [10] has accelerated research in Natural Language Processing (NLP). In fact, pre-training language models on large corpora (which can be thought to be

like large swathes of unlabeled text) has shown to be effective in many downstream tasks. Although earlier efforts mostly utilized general (open) domain data for pre-training [2, 18, 32], limited ability of such PLMs to fine-tune to end tasks on specialized domains such as biomedical or law has prompted efforts in domain-specific pre-training [1, 16].

Particularly, NLP is increasingly being applied in the legal domain, and naturally there have been efforts to develop PLMs for the legal domain. Chalkidis et al. [9] pre-trained *BERT-base* on legal documents from EU, UK and US. This model has been shown to be effective in many downstream legal tasks, such as text classification, summarization and named entity recognition, among others. Again, Zheng et al. [33] pre-trained *BERT-base* on only US case law documents, which was shown to improve performance over US legal datasets such as CaseHOLD. More recently, Henderson et al. [12] pre-trained *BERT-large* on ‘Pile of Law’ (PoL), a huge legal corpus of US and EU documents.¹ Finally, the Lawformer model [31], pre-trained on Chinese legal text, has been designed keeping in mind that legal documents are usually much longer than documents in the general domain.

With the advent of digitization and image processing technologies, a lot of legal data has become readily available in digital format across many countries, like India. This has led to several new studies and technologies being developed on Indian legal data in recent years [3, 4, 13, 20, 24]. Using BERT off-the-shelf for these tasks has been shown to be limited in performance, and although LegalBERT improves upon BERT across many such tasks, there is still scope for improvement in tasks of Indian legal text. This has to do particularly with the differences in the nature of Indian legal texts vis-a-vis EU and US legal texts [3, 13]. Also, due to typographical or scanning errors, there are a lot of instances of missing spaces and punctuation marks, which lead to wrong words. Thus, it is beneficial to pre-train LMs over Indian legal text, allowing the model to observe such patterns during pre-training, so that performance over end-tasks over Indian legal text improves.

In this work, we conduct pre-training experiments with PLMs on a large Indian legal corpus. We collected a large corpus of court judgment documents from the Supreme Court and many High Courts of India, along with a collection of all Central Government Acts (statutory documents). Our dataset is decently positioned in terms of the number of documents (~5.4M) and size (~27GB) as

¹ Although all these three models were named as LegalBERT in the original research papers, we shall address them as LegalBERT [9], CaseLawBERT [33] and PoLBERT [12] respectively, for sake of comprehension.

compared to other pre-training datasets used in the legal domain (see Table 1). We pre-train BERT-based LMs on this corpus and estimate the efficacy of domain-specific pre-training on a number of downstream end-tasks on both Indian legal documents as well as legal documents from other countries/domains.

Our Contributions: To enumerate briefly, our contributions are as follows:

- (i) We re-trained (continued pre-training)² two publicly available pre-trained BERT-based legal PLMs, namely LegalBERT [9] and CaseLawBERT [33] on a large corpus of Indian legal corpus. Both the Indian PLM variants – which we name InLegalBERT and In-CaseLawBERT respectively – outperform their source LMs in terms of perplexity on Indian data.
- (ii) We also pre-trained a BERT-base model from scratch on this Indian legal pre-training corpus, with a custom vocabulary specifically tailored for Indian legal text. We name this model CustomInLawBERT.
- (iii) We compare these three Indian legal PLMs among themselves, as well the original LegalBERT and CaseLawBERT, on several practically important end-tasks, namely, (1) Legal Statute Identification [6, 24], (2) Semantic Segmentation of judgements [4, 5], and (3) Court Judgement Prediction [20]. We perform experiments on different benchmark datasets (based on both Indian and non-Indian legal texts) for these tasks. Our experiments broadly show that InLegalBERT achieves appreciable gains over LegalBERT across all tasks (while the gains are minimal for InCaseLawBERT over CaseLawBERT). We also observe that CustomInLawBERT comes close to the performance of both InLegalBERT and InCaseLawBERT on multiple datasets, despite having been pre-trained for a much lesser number of steps (700k vs. 1.3M vs. 2.3M steps respectively).
- (iv) To understand the inherent differences between the various PLMs, we analyze the model attentions for the Court Judgement Prediction task with a small set of expert-annotated documents (made available in [20]). Comparing model-assigned attention scores with expert-annotated scores shows that the correlation between these scores increases with pre-training on Indian data.

Out of all the pre-trained Language Models (PLMs) that we experiment with, the InLegalBERT (developed in this work) outperforms all other PLMs in almost all the end-tasks that we experimented with. Additionally, comparing with the original works which contributed the respective datasets, InLegalBERT establishes state-of-the-art results on 4 out of 5 datasets. These results show that further pre-training on Indian legal data helps in improving performance over English-language legal text datasets from not only India but from other countries as well.

All the three models are publicly available on HuggingFace.³ The code for pre-training are also available on GitHub.⁴

2 RELATED WORK

Transformer-based PLMs: After the introduction of the self-attention based Transformer mechanism [27], we have seen the widespread

development of large-scale, pre-trained language models. Devlin et al. [10] first introduced BERT, a multi-layer transformer architecture for language modeling. They used the Masked Language Modeling (MLM) objective [26] to mask some tokens randomly in the text, to effectively train a deep bidirectional language model on general domain corpora, including English Wikipedia. They also used Next Sentence Prediction (NSP) objective to sample pairs of contiguous sentences. Their proposed approach of transfer learning differed from prior works in the sense that most earlier works used to transfer pre-trained embeddings for the end tasks, whereas BERT proposed transferring the pre-trained model instead.

The unprecedented success of BERT on a range of downstream NLP tasks such as GLUE [28] and SQUAD [25] prompted research in transformer based PLMs. RoBERTa [18] used a much larger pre-training corpus and did away with the NSP objective, going for larger batch sizes instead. XLNet [32] modified the pre-training objective to Permutation Language Modeling. Longformer [2] introduced a global-local attention mechanism to deal with longer sequences. All these models were shown to outperform BERT on many general domain downstream tasks. However, these PLMs, pre-trained on general domain data, often show limited performance for tasks in specialized domains such as biomedical and law, leading to efforts in pre-training such models on domain-specific data, e.g., BioBERT [16] and SciBERT [1].

PLMs for the legal domain: In recent years, a lot of legal tasks and datasets have been released for multiple domains/countries, such as US, EU, China, India and so on. This has led to use of pre-trained transformer based models such as BERT [10] on these datasets. Legal text being very unique, there is significant scope of improvement over models such as BERT pre-trained on the general domain. There have been multiple efforts to pre-train transformers on the legal domain: (i) Chalkidis et al. [9] pre-trained BERT-base on EU and UK legislation and court documents from the US, European Court of Justice (ECJ) and European Court of Human Rights (ECtHR), and released the LegalBERT model; (ii) Zheng et al. [33] proposed CaseLawBERT, pre-trained on a corpus of US case law documents and contracts; (iii) Henderson et al. [12] prepared a huge corpus of US, Canada and EU documents (not just case law), called *Pile of Law*, and trained BERT-large on the same to yield the PoLBERT model; (iv) Xiao et al. [31] released Lawformer, a Longformer [2] based model pre-trained on Chinese legal text. The details of the pre-training datasets used by these models are available in Table 1.

However, no work has compared the performances of these different PLMs in a common setting. In addition, it has not been explored how these existing PLMs can be utilized to develop PLMs for legal text of other domains/countries. We attempt to fill these gaps in this paper, with a case-study on Indian legal text.

3 DATA COLLECTION

For pre-training, we collected court case documents from different courts of India, specifically, the Supreme Court, 26 different High Courts and 2 different District Courts (total of 29 courts). We used publicly available sources such as official websites of these courts (e.g., the website of the Indian Supreme Court: <https://main.sci.gov.in/>), the erstwhile website of the Legal Information Institute of India, the popular legal repository <https://www.indiankanoon.org>,

²We use the terms ‘re-train’ and ‘continue pre-training’ or ‘further pre-training’ interchangeably throughout the text.

³InLegalBERT – <https://huggingface.co/law-ai/InLegalBERT>

InCaseLawBERT – <https://huggingface.co/law-ai/InCaseLawBERT>

CustomInLawBERT – <https://huggingface.co/law-ai/CustomInLawBERT>

⁴<https://github.com/Law-AI/pretraining-bert>

Model	Data Source	#Documents	Size (GB)
LegalBERT	EU, UK, US legislation, cases from ECJ, ECtHR	350K	12
CaseLawBERT	Harvard Case Law (based on US federal and state courts)	3.4M	37
PoLBERT	Legal Analyses, Court Opinions, Government Publications, Contracts, Statutes, Regulations, and more from US and EU	10M	256
Lawformer	China Judgment Online (based on cases from Chinese Courts)	22.7M	84
InLegalBERT, InCaseLawBERT, CustomInLawBERT	Cases from Indian Supreme Court and High Courts	5.4M	27

Table 1: Comparison of pre-training datasets used in various works in the legal domain. The last row is about the Pre-trained Language Models (PLMs) developed in this work.

and so on. Additionally, we collected 1,113 Central Government Acts, which are the documents codifying the laws of the country. Each Act is a collection of related laws, called Sections. These 1,113 Acts contain a total of 32,021 Sections.

In total, we collected around 5.4 million documents, comprising of around 27GB raw text. A brief comparison of our pre-training dataset with those used in prior works is given in Table 1.

The court cases in our dataset range primarily from 1950 to 2019 (India became a republic in 1950), although there are a few cases dating as far back as the 1800s. These cases belong to all legal domains, such as Civil, Criminal, Constitutional, and so on. The temporal distribution of cases in our dataset is heavily skewed towards the recent decades. This skew is natural, since cases have risen exponentially with India’s growing population, as well as the increasing application of digital techniques in the judicial setup for keeping records.

Pre-Processing the documents: As discussed in Section 1, Indian legal text is known to be noisy. There are many odd artefacts in the text, such as sequences of dashes or lines, page numbers and meta-data in random positions of the text, etc. Most of such anomalies have crept into the text due to digitization of hand-written documents (which we verified manually by comparing older documents with newer ones, especially post 2010). We remove such artefacts during pre-training using Regular Expression matching.

4 PRE-TRAINING SETUP

We now describe our approach for developing a PLM for Indian legal text. We conduct different types of independent pre-training experiments –

- by starting with the two existing pre-trained models – LegalBERT [9] ⁵ and CaseLawBERT [33] ⁶ – and continuing the pre-training on our Indian dataset (Re-training). We name the two final models (after re-training on the Indian dataset) as InLegalBERT and InCaseLawBERT, respectively.
- by training a BERT-base-uncased model from scratch, and utilizing a custom vocabulary trained on our data. We name this model as CustomInLawBERT.

4.1 Custom Vocabulary for Indian Legal Data

The language of Indian legal documents is quite different from legal documents of other countries, even though all are written in English. One of the major reasons for this difference is the frequent use of local terms like “daroga” (constable) or “lathi” (stick), or local

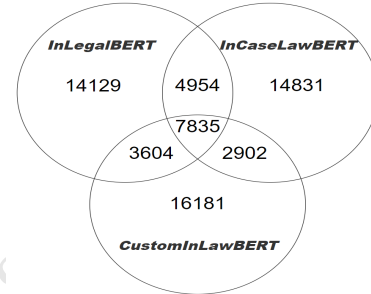


Figure 1: Comparison of vocabularies of the three PLMs – InLegalBERT (vocabulary same as that of LegalBERT), InCaseLawBERT (vocabulary same as that of CaseLawBERT) and our custom vocabulary for Indian legal text.

salutations like “Shri” (Mr.) or “Smt.” (Mrs.), and so on. These factors motivate the need of using a custom vocabulary tailored for Indian legal documents, which we follow for the experiment that involves pre-training from scratch (for developing CustomInLawBERT).

We use the WordPiece algorithm [30] employed by BERT for training our custom tokenizer. We choose 10% of the training data to train this tokenizer. We keep all the default parameters used for training the BERT tokenizer (including the vocabulary limit of 30,522 tokens).

Figure 1 shows a quantitative comparison between the vocabularies of InLegalBERT (same as that of LegalBERT), InCaseLawBERT (same as that of CaseLawBERT) and our custom vocabulary. There is a significant number of tokens that are common to all three vocabularies, which represent common English words. It is also interesting to note that the number of tokens common to two vocabularies but not to the third is significantly lower than the tokens common to all. On the other hand, each of the three vocabularies has a substantial number of unique tokens which are *not* contained in the other vocabularies. Also note that there is a slightly higher degree of overlap between the vocabularies of InLegalBERT and InCaseLawBERT, as compared to their overlap with our custom Indian vocabulary. This highlights the presence of unique words in Indian legal data.

4.2 Creating training and testing examples for pre-training

We split the corpus into train and test splits. The test split is used to evaluate the pre-training quality using the perplexity metric (details later in this section). The case documents were distributed randomly into the two splits in the ratio of 9:1, while maintaining

⁵<https://huggingface.co/nlpauueb/legal-bert-base-uncased>

⁶<https://huggingface.co/zhucia/legalbert>

the distribution of document lengths (in terms of no. of sentences) across both splits.

To create individual training and testing examples, we first divided each document D into multiple chunks $\{C_1, C_2, \dots\}$. For documents in both train and test dataset, we divided each document into a set of equal-sized disjoint chunks of contiguous text, disregarding sentence boundaries. Each chunk is a unique training/testing example. This strategy helps us train on longer sequences, while reducing training time [10]. Chunking also does not affect the two pre-training objectives — MLM and NSP. Following this strategy, we end up with a total of *21.6M training examples*, and *2.4M testing examples*.

4.3 Training Objectives

We use the standard Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) training objectives that were also used in pretraining BERT [10]. MLM involves masking some tokens of each input during encoding, and the model should predict the masked tokens based on the context. Under NSP, a pair of inputs is passed to BERT, and the model is required to predict whether the second input follows the first in the document sequence. We use both *dynamic masking* for MLM [18] and *dynamic sampling* for NSP, meaning that the choice of masked words (MLM) and negative samples (NSP) are *not* fixed beforehand in a static way; rather, it is dynamically chosen when a mini-batch is being built. For each training example $C_i \in D$ (which is a chunk of a document, as stated above), we either pair it up with its consecutive chunk C_{i+1} (positive sample) or randomly choose any other chunk from the same document $C_j, j \neq i, i+1$ (negative sample) in a 1:1 ratio. This gives us the sequence pair for the NSP task.

Then, for each such pair of chunks (C_i, C_j) , we mask some tokens by following the masking policy described by Devlin et al. [10]. Specifically, for each input pair, we randomly choose 15% tokens across both inputs as mask candidates. Models like BERT use a tokenization scheme which can split words into sub-word units. We use whole-word masking, i.e., ensuring that candidate tokens are chosen in a manner where all sub-word tokens of a real word are chosen, or none at all. Out of all such candidate tokens, we replace them with the [MASK] token 80% of the time, keep them unchanged 10% of the time, and replace them with a random token in the vocabulary 10% of the time.

The masked sequence pair (C'_i, C'_j) is passed to the BERT model along with the special tokens [CLS] and [SEP], yielding a hidden embedding for each token t_k in the entire sequence pair.

$$Input \leftarrow [CLS] C_{i1} C_{i2} \dots C_{i|C_i|} [SEP] C_{j1} C_{j2} \dots C_{j|C_j|} [SEP]$$

$$\mathbf{h}_k \leftarrow BERT(t_k) \quad \forall t_k \in Input$$

From the BERT outputs, we pass the hidden embeddings of every token through a linear classifier for MLM.

$$P(\hat{y}_k^{(MLM)} = v) \leftarrow softmax(ff_{MLM}(\mathbf{h}_k)) \quad \forall v \in V$$

where V represents the vocabulary, ff represents a linear layer.

For NSP, we use the BERT pooler output, which is basically the output embedding of the [CLS] token further passed through a feedforward network with *tanh* activation. The pooler output is

passed through another linear classifier for NSP.

$$P(\hat{y}^{(NSP)} = c) \leftarrow softmax(ff_{NSP}(tanh(ff_{pooler}(\mathbf{h}_{[CLS]}))))$$

where $c \in \{0, 1\}$ represents whether the second input follows the first or not. Both MLM and NSP are trained using the Categorical Cross Entropy Loss. For MLM, the loss is calculated *only* over the masked tokens.

4.4 Implementation Details

Both the existing PLMs (used for re-training) as well as our custom PLM (trained from scratch) are based on the BERT-base-uncased model (12 layers, 768 hidden dimensionality, 12 attention heads, 110M parameters). Due to the constraints on input lengths of BERT-based models, all chunks are limited to 512 tokens (including special [CLS] and [SEP] tokens). To protect most information from truncation, we chunk training documents into segments of 254 tokens each. Thus, while sampling pairs for NSP, we ensure the total length of the sequence pair is limited to 512. All experiments are performed on a single Nvidia RTX A6000 (48 GB VRAM).

For re-training, we train for 300K optimization steps, which is roughly equivalent to 4 full epochs over the training split. The model being trained from scratch naturally requires more training steps to make it comparable to the re-training setups, and thus we train this model for 700K optimization steps. It is important to note that our setup is still quite limited compared to prior works, which employed a larger number of pre-training steps for both re-training as well as pre-training from scratch. For instance, Chalkidis et al. [9] performed 500k pre-training steps for re-training BERT, and 1M steps for a model trained from scratch. Zheng et al. [33] performed 1M steps when re-training BERT and 2M steps for pre-training from scratch. At the end of our pre-training effort, we end up with models that have been trained for vastly different total no. of steps: InLegalBERT (total 1.3M steps), InCaseLawBERT (total 2.3M steps), CustomInLawBERT (total 700k steps).

We used the default AdamW optimizer [19] with initial learning rate of $5e-5$. Apart from these, we kept all other hyper-parameters to their default values (i.e., values used by Devlin et al. [10] for pre-training BERT). We also used other techniques to speed up training, such as fp16 training, pinning the CPU memory for GPU transfer, and using 8 CPU workers for preparing the batches. In our setup, each re-training experiment took approximately 20 days (300K steps), while training from scratch took approx. 50 days (700K steps). Testing using perplexity took approx. 1 day. We have had to use limited no. of pre-training steps (as discussed above) mainly because it takes such a lot of time to perform one pre-training experiment on a modest computational setup as ours.

For pre-training (and eventually fine-tuning) tasks, we used the datasets package [17] by HuggingFace (HF) for dealing with loading, tokenizing and formatting. We also used the transformers package [29] by HF to load the models and run training and evaluation tasks. We used our own code since the default BERT pre-training code provided by Devlin et al. [10] do not perform dynamic MLM and NSP. Our entire code base is built on PyTorch [23].

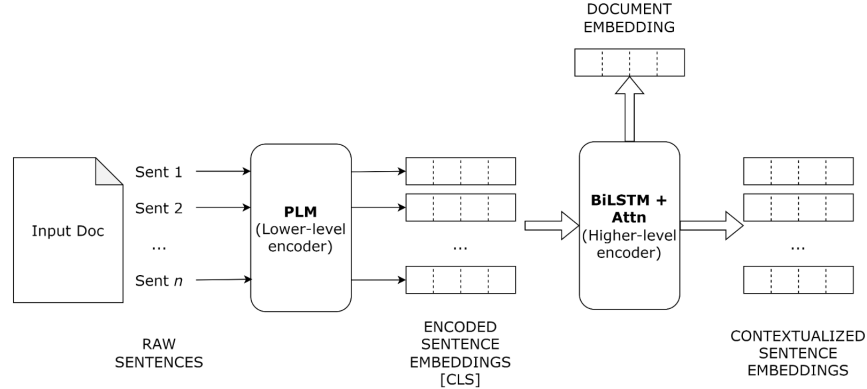


Figure 2: The HierBERT architecture that is used for all end-tasks. We experiment with different versions of this architecture which use different PLMs (e.g., LegalBERT, InLegalBERT, InCaseLawBERT, CustomInLawBERT, etc.) for the lower-level encoder. The rest of the architecture is kept the same in all versions, for a fair comparison among the different PLMs.

Model	Perplexity
BERT	25.7600
LegalBERT	7.1331
CaseLawBERT	11.1798
InLegalBERT	5.2547
InCaseLawBERT	8.7824
CustomInLawBERT	11.7549

Table 2: Quality of pre-training measured via perplexity. All perplexity values reported over our test set (10% of the Indian legal text corpus).

4.5 Evaluation of pre-training

To get an idea of the quality of pre-training, we evaluate the pre-trained models on the test set (10% of the corpus, as stated earlier). We evaluate the language modeling capabilities of the models using the *perplexity* metric. As mentioned previously, LMs, in general, learn to estimate the probability of a sequence of tokens. The perplexity metric scores the model based on these probabilities. For models that use MLM, the models can be scored based on the probabilities assigned to the masked tokens. Mathematically, perplexity can be calculated as the exponentiation of the average Negative Log-Likelihood (NLL). Thus, a lower value of perplexity indicates a better score for the LM. In our case, we can directly use the loss only for the MLM task ($\mathcal{L}_{MLM}$) — $Perplexity = e^{\mathcal{L}_{MLM}}$

Table 2 shows the perplexity values of the various models over our test set. We observe lower perplexities for the Indian variants InLegalBERT and InCaseLawBERT, as compared to the original PLMs. In particular, InLegalBERT seems more adaptable to the Indian scenario (lower perplexity) as compared to InCaseLawBERT. CustomInLawBERT does not achieve as low perplexities as the other legal PLMs, but it still does better than BERT.

5 APPLICATION ON END-TASKS

We now apply (fine-tune) the pre-trained models on several practically important and benchmark end-tasks over both Indian legal text as well as legal text from other domains/countries. While these end-tasks may eventually require more complex architectures, all

such architectures contain an inherent BERT-based encoder module for encoding the text either in parts or full. We try out the different PLMs, used in this study, as this encoder module, for a fair comparison.

We chose the standard BERT-base [10], the original LegalBERT [9] and the original CaseLawBERT [33] as baselines. The latter two models are based on the same architecture as BERT-base.⁷

5.1 The fine-tuning architecture used for all end-tasks

For all fine-tuning experiments, we used the general HierBERT architecture [7] with task-specific classification heads on top, as shown in Figure 2. We experiment with different versions of this architecture which use different PLMs (e.g., LegalBERT, CaseLawBERT, InLegalBERT, InCaseLawBERT, CustomInLawBERT, etc.) for the lower-level encoder. The rest of the architecture is kept the same in all versions, for a fair comparison among the different PLMs.

Since the training examples for all end tasks consist of long documents (much beyond the sequence limit of 512 tokens that BERT can accommodate), we divide each document into sentences/chunks⁸, and represent the document as a sequence of sentences/chunks.

$$D = \{C_1, C_2, \dots, C_{|D|}\}, C_i = \{t_{i1}, t_{i2}, \dots, t_{i|C_i|}\}, t_{i1} = [\text{CLS}]$$

HierBERT consists of a lower-level BERT encoder, which is used to encode individual sentences/chunks.

$$\mathbf{h}_{i1}, \mathbf{h}_{i2}, \dots, \mathbf{h}_{i|C_i|} \leftarrow \text{BERT}(\{t_{i1}, t_{i2}, \dots, t_{i|C_i|}\})$$

with the [CLS] embedding ($\mathbf{h}_{i1}$) representing the entire span. The sequence of such [CLS] embeddings across sentences/chunks are then fed to a higher-level encoder, which in our case is an LSTM module with an Attention Head. The LSTM generates a sequence of context-aware embeddings (respective to each sentence/chunk).

$$\mathbf{h}'_1, \mathbf{h}'_2, \dots, \mathbf{h}'_{|D|} \leftarrow \text{LSTM}(\{\mathbf{h}_{11}, \mathbf{h}_{21}, \dots, \mathbf{h}_{|D|1}\})$$

⁷For the sake of fair comparison, we did *not* choose PolBERT [12] as a baseline since it is based on BERT-large, which is inherently more powerful.

⁸Some datasets contain documents already divided into sentences, for others we perform chunking

Encoder Module	ILSI dataset			ECtHR-B dataset		
	mP	mR	mF1	mP	mR	mF1
BERT	82.12	49.07	59.11	77.50	69.31	72.95
LegalBERT	83.98	53.83	63.89	80.85	70.76	75.09
CaseLawBERT	82.89	54.72	64.53	82.37	66.45	72.87
InLegalBERT	82.42	55.16	64.58	83.93	71.41	75.88
InCaseLawBERT	81.07	55.64	64.44	77.35	69.45	72.86
CustomInLawBERT	82.48	54.46	64.29	79.64	67.09	72.56
Best result in original paper	27.90	31.32	28.45	-	-	74.70

Table 3: Performance over the ILSI and ECtHR-B datasets for Legal Statute Identification task. All reported values are macro-averaged and in terms of percentage. Difference in m-F1 results for InLegalBERT and all other variants is statistically significant based on the Wilcoxon Signed Rank Test.

The Attn module produces a single embedding for the entire document from these context-aware sentence/chunk embeddings.

$$\mathbf{h}'_D \leftarrow \text{Attn}(\{\mathbf{h}'_1, \mathbf{h}'_2, \dots, \mathbf{h}'_{|D|}\})$$

For some tasks, we require a single document-level representation of the entire input, while for others, we require representations for every sentence/chunk.

For every experiment, we also compare results obtained using our models with the best results reported in the original papers that introduced the datasets we are working with. The Wilcoxon Signed Rank Test has been shown to be effective for most NLP setups [11]; we employ it to verify statistical significance of the best results.

5.2 Legal Statute Identification (LSI)

In countries that follow some notion of the Civil Law System, there are written laws (statutes) that guide jurisdiction. In court cases of such systems, one or more of these laws are cited, based on their relevance to the facts (evidences) of the case. The task of Legal Statute Identification (LSI) aims to automatically identify the relevant statutes (i.e., the statutes that may have been violated) given the facts of the case [8, 21, 24]. LSI is widely studied, and is most commonly modeled as a multi-label text classification task.

5.2.1 Datasets. We use two different datasets for this task:

(i) **ILSI:** The Indian Legal Statute Identification (ILSI) Dataset is based on criminal case documents from India [24]⁹. The dataset consists of 65K examples (derived from criminal court cases from the Supreme Court and 6 High Courts of India), split into train, dev and test splits, and a target set of 100 statutes (classes/labels) from the Indian Penal Code. All 100 statutes (class labels) are part of the Indian Penal Code, which contains the defined laws for most criminal procedures in India. More details of the dataset are available in [24].

(ii) **ECtHR-B:** This is the dataset used in the ECtHR Task B from the LexGlue benchmark suite [6]¹⁰. The dataset consists of 11K examples (facts from cases argued in the European Court of Human Rights), split into train, dev and test. The label set consists of 10 articles (classes/labels) from the European Convention of Human Rights, which mainly contain provisions on human rights issues. Each fact is mapped with the allegedly violated articles from this label set.

5.2.2 Implementation. We used the vanilla HierBERT architecture with a feed-forward network (sigmoid activation) over the document embeddings for the final classification. Both ILSI and ECtHR-B datasets provide the facts segmented into natural sentences. We truncated each sentence (after tokenization) to a maximum of 128 tokens, and chose the first 128 sentences for each document, and these sentences are fed to HierBERT.

All intermediate embeddings have dimensionality of 768. We experimented with multiple batch sizes {32, 64, 128} and found a batch size of 64 to be the most suitable¹¹. The model was trained for 25 epochs (with early stopping) with Weighted Binary Cross Entropy Loss, with the label weights being inversely proportional to the label frequency. We also experimented with a uniform initial learning rate across all layers of 5e-5, as well as different learning rates for different layers (1e-3 for the upper encoder of HierBERT and the classification layer, 1e-5 for the lowest BERT layer) and found the latter more suitable. It took around 40 hrs to run a single experiment for the ILSI dataset and 8 hrs for the ECtHR-B dataset on a single RTX A6000 GPU.

5.2.3 Results. We report Macro-Precision (mP), Macro-Recall (mR) and Macro-F1 (mF1) metrics for comparing the performance of different PLMs (the architectures are identical in all other aspects). The results over both the datasets are shown in Table 3.

For the ILSI dataset, the vanilla BERT model, which was pre-trained over only general domain data, performs the poorest across all metrics. LegalBERT leverages the pre-trained knowledge of legal data (albeit of a different country), and obtains massive gains over BERT. It also obtains the highest Precision score. However, CaseLawBERT does even better in terms of F1 over LegalBERT. This scenario changes for the ECtHR-B dataset, where we see LegalBERT outperforming CaseLawBERT by some margin. This might be due to the fact that although CaseLawBERT has been trained for more steps on a much larger dataset of US legal documents (helping it to generalize better for the Indian domain), LegalBERT has been trained on ECtHR documents, and thus performs better for the ECtHR-B dataset.

Injecting India-specific legal knowledge further boosts results for InLegalBERT across both datasets. Although Precision reduces slightly from the vanilla LegalBERT on the ILSI dataset, there are significant improvements for both Recall, and more crucially, F1 score. There is an outright improvement achieved by InLegalBERT

⁹<https://github.com/Law-AI/LeSiCiN>

¹⁰https://huggingface.co/datasets/lex_glue

¹¹By most suitable, we refer to the hyper-parameter setting that produces maximum performance on the respective validation set

Encoder Module	ISS dataset			UKSS dataset		
	mP	mR	mF1	mP	mR	mF1
BERT	67.56	64.21	64.41	64.28	58.51	59.54
LegalBERT	71.81	65.70	67.32	62.87	60.22	60.03
CaseLawBERT	70.39	68.53	68.03	61.61	60.23	59.68
InLegalBERT	71.65	68.36	68.98	63.93	60.95	61.54
InCaseLawBERT	72.20	68.21	68.80	64.68	59.82	60.85
CustomInLawBERT	70.9	66.91	67.25	63.28	60.41	60.62
Best result in original paper	83.96	80.98	82.08	65.30	58.70	60.00

Table 4: Performances for the Semantic Segmentation task. All reported values are macro-averaged and then averaged across 5-folds (cross-validation) and in terms of percentage. Difference in m-F1 results for InLegalBERT and all other variants (best fold model) is statistically significant based on the Wilcoxon Signed Rank Test.

across all metrics for the ECtHR-B dataset. However, InCaseLawBERT shows a slight drop in F1 for both datasets with increase in Recall. This shows that LegalBERT is more adaptable to further pre-training on data from other countries. CustomInLawBERT shows comparable performance to both InLegalBERT and InCaseLawBERT on ILSI despite being pre-trained for much lesser steps (700k vs. 1.3M vs. 2.3M steps respectively), although CustomInLawBERT uses an Indian-specific vocabulary, which is a better fit for the text of ILSI. The same vocabulary is not suited for the text of ECtHR-B, and thus the performance for CustomInLawBERT is much lower than InLegalBERT. The difference in results of Table 3 are statistically significant based on the Wilcoxon Signed Rank Test (choice of test based on the observations of Dror et al. [11]).

Comparison with the original works: The original work [24] that introduced the ILSI dataset, employed an architecture (called LeSICiN) which is a hybrid of text-based and graph-based encodings. In particular, they used a Sent2vec encoder [22] (a relatively shallow pre-training approach based on FastText) for encoding text, and reported the best performance of macro-F1 28%. The models reported in this paper outperform the best result reported in [24] massively on the ILSI dataset (Table 3). We think that transformer-based pre-training, along with some fine-tuning experimental setups, such as different initial learning rates to different kinds of layers, have helped to optimize the models effectively.

On the other hand, Chalkidis et al. [6] used a very similar architecture to ours for the ECtHR-B dataset, i.e., HierBERT, but with a 2-layer transformer on top instead of LSTM. They also reported results using LegalBERT and CaseLawBERT encoders (m-F1 of 74.7% and 70.3% respectively), which we beat with our architecture (Table 3). We had also experimented with this version of the HierBERT architecture (with a 2-layer transformer on top) for both datasets, and we observed that the performance was slightly less on ECtHR-B and significantly less on ILSI as compared to the HierBERT version with LSTM + Attn on top. Thus, we only report the results for HierBERT with LSTM + Attn on top. Our model with InLegalBERT obtains higher performance over all results reported in Chalkidis et al. [6] for ECtHR-B – the macro-F1 obtained by our model is 75.88% compared to the best value of 74.7% reported in [6].

5.3 Semantic Segmentation of case judgements

Legal case judgements are composed of different functional or rhetorical segments such as *Facts*, *Issue*, *Ratio for the ruling*, *Ruling*, etc. [4]. Demarcating these segments can be helpful, since many applications require analyzing some specific segments while not

considering the rest, e.g., we need to extract only the facts for tasks such as LSI. Again, since legal case judgements are very long, law professionals may want to read only certain segments such as only the legal issues in a case. Legal judgements in many countries do not have clearly demarcated boundaries for the above segments, and thus developing automated methods for semantic/rhetorical segmentation is a very widely researched task. The task can be considered as a sequence labeling problem, where the entire document can be considered as a sequence of sentences with each sentence being assigned a particular semantic segment.

5.3.1 Datasets. We use two datasets for this task as well:

(i) **ISS:** We use the dataset provided by Bhattacharya et al. [4] and call it the Indian Semantic Segmentation dataset (ISS).¹² The ISS dataset has 50 documents from the Supreme Court of India, with each sentence marked with one of seven semantic/rhetorical segment labels – *Facts*, *Arguments*, *Statutes*, *Precedents*, *Ratio Decidendi*, *Ruling by Lower Court* and *Ruling by Present Court*. In total, there are 9,308 sentences marked with one of the labels stated above.

(ii) **UKSS:** We also use the dataset based on UK Supreme Court cases provided by Bhattacharya et al. [5], calling it the UK Semantic Segmentation dataset (UKSS).¹³ The UKSS dataset has 50 documents from the Supreme Court of the UK, segmented into 18,155 sentences, and each sentence is annotated with one of the same seven semantic/rhetorical as the ISS dataset.

5.3.2 Implementation. For this task, we use the HierBERT architecture with Conditional Random Field (CRF) classifier [15] over the contextualized sentence embeddings, since CRFs are known to work well for sequence labeling tasks. We limit the sentence length to 128 tokens, but include all sentences in a document. All intermediate embeddings are of size 768. We experimented with multiple batch sizes {1,2,4} and found a batch size of 2 to be the most suitable. The model was trained for 25 epochs (with early stopping) with Negative Log Likelihood Loss. We also experimented with a uniform initial learning rate across all layers of 5e-5, as well as different learning rates for different layers (1e-3 for the upper encoder of HierBERT and the CRF layer, 1e-5 for the lowest BERT layers) and found the latter more suitable. We used 5-fold cross-validation for both datasets. It took around 5 hrs to complete all five folds of a single experiment on a single RTX A6000 GPU.

¹²<https://github.com/Law-AI/semantic-segmentation>

¹³Dataset available upon request to the first author

5.3.3 Results. The average results (across 5 folds) for the Semantic Segmentation task on both ISS and UKSS datasets are shown in Table 4. For this task, the trends are very similar for both datasets. Taken off-the-shelf, both LegalBERT and CaseLawBERT obtain gains over BERT in terms of F1 score across both datasets, however the difference is larger for ISS than UKSS. LegalBERT does better than CaseLawBERT for the UK dataset and vice-versa for the Indian dataset (see Table 4). This observation is similar to what we observed in the LSI task as well (see Section 5.2). On further re-training on Indian data, performance of both LegalBERT and CaseLawBERT increases, however the margins are lesser for CaseLawBERT on both datasets. CustomInLawBERT slightly underperforms all the other legal PLMs, most possibly due to much lesser no. of steps in pre-training, as discussed in Section 5.2. InLegalBERT beats all variants in terms of macro-F1 on both datasets (statistically significant based on Wilcoxon Signed Rank Test).

Comparison with the original works: None of the transformer-based models stated above are able to perform as well as the best method in [4] for the ISS dataset, which has the same broad architecture as described above, but uses a Sent2vec encoder [22] (based on FastText) trained on Indian Supreme Court documents instead. This Sent2vec-based model has a very high performance across all metrics ($> 80\%$ m-F1) as reported in [4]. We suspect this could be due to the very small size of the ISS dataset (which may not be sufficient to fine-tune transformer-based models effectively). For the UKSS dataset, Bhattacharya et al. [5] fine-tuned both BERT and LegalBERT under the same HierBERT-CRF setup as we are using, but had frozen all but the top 2 layers of BERT. Under our setup (fine-tuning all layers), we achieve similar performance as compared to the result reported by Bhattacharya et al. [5] for LegalBERT encoder, which is 60% m-F1 (see Table 4). Using InLegalBERT further improves on this performance to achieve 61.54% m-F1 which is higher than the best result reported in [5].

5.4 Court Judgment Prediction

Automatically predicting the outcome of court cases has been widely studied in recent years [14, 20, 34]. This has mostly been done via the task of Legal Judgment Prediction (LJP), where, given the facts of a case, we are to predict the laws/statutes violated, applicable charges and terms of penalty. Along these lines, the CJPE (Court Judgement Prediction and Explanation) task, introduced by Malik et al. [20], aims to predict the final decision of the court, based on the rest of the case judgement (i.e., the input is the case judgement document with the final decision removed). In the Indian scenario, a court case usually consists of one or multiple appeals/claims filed by the appellant against the respondent, and the judge needs to provide an ‘accept’ or ‘reject’ decision for each of these claims. Thus, this can be modeled as a binary text classification task.

5.4.1 Dataset. The ILDC family of datasets, introduced by Malik et al. [20] contain different types of training data.¹⁴ Among them, the ILDC-multi dataset consists of approx. 35K Supreme Court of India case documents each having multiple claims/appeals; a single label is assigned for each document – ‘accept’ if at least one

Encoder Module	mP	mR	mF1
BERT	71.31	70.98	71.14
LegalBERT	79.85	78.49	78.21
CaseLawBERT	82.62	82.42	82.38
InLegalBERT	83.43	83.15	83.09
InCaseLawBERT	83.05	82.82	82.77
CustomInLawBERT	80.38	79.73	79.60
Best result in original paper	77.32	76.82	77.07

Table 5: Performance over the ILDC-multi dataset for the Court Judgement Prediction task. All values are in terms of percentage. Difference in m-F1 results for InLegalBERT and all other variants is statistically significant based on the Wilcoxon Signed Rank Test.

appeal in that case document is accepted, ‘reject’ otherwise. For our current experiment, we consider the ILDC-multi dataset.

5.4.2 Implementation. We use the HierBERT architecture which was also used by Malik et al. [20] for their results, with a linear layer (sigmoid activation) on top. The text of the ILDC dataset is not pre-segmented into sentences (unlike all the other datasets described previously), and thus we chunk the entire text (after tokenization) into contiguous, non-overlapping groups of 128 tokens, and passed to the HierBERT model. We choose the last 128 chunks (based on the observation by Malik et al. [20] that the end portion of documents contains information that can be helpful for CJPE). All intermediate embeddings are of size 768. We experimented with multiple batch sizes {16,32,64} and found a batch size of 32 to be the most suitable. The model was trained for 25 epochs (with early stopping) with Binary Cross Entropy Loss. We also experimented with a uniform initial learning rate across all layers of $5e-5$, as well as different learning rates for different layers ($1e-3$ for the upper encoder of HierBERT, $1e-5$ for the lowest BERT layer) and found the latter more suitable. It took around 30 hrs to complete a single experiment for the ILDC dataset on a single RTX A6000 GPU.

5.4.3 Results. For this task, we again use the vanilla HierBERT architecture with a feed-forward network (sigmoid activation) over the document embedding, having a single output unit as the final classification layer. We report macro-P, macro-R and macro-F1 metrics for evaluating the different PLMs in Table 5. Performance of both LegalBERT and CaseLawBERT improve on re-training over Indian legal data; however, the difference between LegalBERT and InLegalBERT is much greater. This is consistent with our observations for LSI and Semantic Segmentation that LegalBERT is more adaptable to re-training on Indian data as compared to CaseLawBERT. CustomInLawBERT only manages to outperform the vanilla BERT and LegalBERT, falling significantly shorter than both InLegalBERT and InCaseLawBERT.

Comparing with the original paper: All 5 legal PLMs outperform the best result reported by Malik et al. [20], which used the same architecture as ours but employed XLNet [32] as the lower encoder. This is possibly due to the legal knowledge accrued by the PLMs we used, given that Malik et al. [20] did not use any legal PLM based encoder. The m-F1 values achieved by InLegalBERT is statistically significantly higher than that of all other models and the results in the original papers, based on the Wilcoxon Signed Rank Test.

¹⁴<https://github.com/Exploration-Lab/CJPE>

5.5 Analysis of Explainability of Models

Explainability of automated methods is extremely crucial in the legal domain, to be accepted as authentic by both legal professionals and laymen. For domain-specific tasks like Court Judgment Prediction, this is even more important, since the standard output of the model is just a ‘yes’/‘no’ decision. For instance, it is important to understand if the model is basing its predictions over the appropriate portions of the input text. To this end, we now check the agreement of the various fine-tuned models (that use the different PLMs) with the opinions of domain experts for the CJPE task.

5.5.1 Dataset. The ILDC dataset for CJPE [20] includes a small set of 56 documents (taken from the test set), where *five legal experts have manually annotated the spans of text (character-level spans) which are crucial for understanding the final judgment.* We make use of this set, called ILDC-expert for understanding the predictions of the model, which contains five different sets of annotations, corresponding to each expert, per document.

5.5.2 Method. We now describe how we check the agreement of the models with the annotations by the legal experts.

Weightage assigned by fine-tuned models: As discussed in Section 5.4.2, we divide the entire text of each document D into equal-sized chunks and feed these in hierarchical fashion to the models, both during training and testing. To understand the relative weightage placed by a fine-tuned model M over different parts of the text for building the final document representation, we can use the attention scores generated by the upper (BiLSTM + Attn) encoder (shown in Figure 2). For a document D , the upper encoder of M generates a probability distribution $\{p_{D,i}^M\}$ over all chunks $C_{D,i}^M$ of D ($i \in [1, n]$, where n is the total number of chunks). Note that these chunks (of tokens) are not universal – they depend on the tokenizer used, and thus, depend on the model M .

Weightage assigned by experts: For each document D , an expert $\mathcal{E}$ provides one or more contiguous gold-standard character spans for the entire document, marking the parts of the text that guide the final decision of the court. For comparing these annotations of expert $\mathcal{E}$ with the attention weightage generated by a model M , we need to find some notion of chunk-level gold-standard importance based on the annotated spans (i.e., some estimate of which chunks would have been deemed important by experts).

We apply the following procedure to find the gold-standard, chunk-level importance scores of expert $\mathcal{E}$ with respect to model M . For every document D ,

- (i) We tokenize the entire text of D into a sequence of tokens $\mathcal{T}_D^M$ and map the character-level spans to token-level spans, based on the tokenizer of M . This effectively gives us a mapping $\mathcal{R}^{\mathcal{E}} : \mathcal{T}_D^M \rightarrow \{0, 1\}$, from each token $t \in \mathcal{T}_D^M$ to a binary value of 0 (if t is not part of a span that was annotated by the expert) or 1 (if t is part of an annotated span).
- (ii) For each chunk $C_{D,i}^M \in D$, we can count the number of annotated tokens (a token t is annotated iff $\mathcal{R}^{\mathcal{E}}(t) = 1$) inside it. Dividing this by the total number of annotated tokens (by expert $\mathcal{E}$) in the whole document D , gives us the gold-standard distribution $\{q_{D,i}^{M,\mathcal{E}}\}$ of importance over the chunks, for each chunk $C_{D,i}^M \in D$.

Model	E1	E2	E3	E4	E5	Average
LegalBERT	47.4	44.7	44.7	42.8	44.8	44.9
CaseLawBERT	55.2	50.5	48.1	47.1	53.6	50.9
InLegalBERT	44.4	43.5	43.8	41.1	45.4	43.6
InCaseLawBERT	40.7	42.4	43.1	40.9	40.7	41.6
CustomInLawBERT	43.7	47.3	47.0	43.9	40.5	44.5

Table 6: KL Divergence of the model-assigned chunk attentions for LegalBERT, InLegalBERT and CustomInLawBERT w.r.t the probabilities of annotated tokens in every chunk. Columns represent each of the 5 legal experts who have provided the annotations.

Thus, following the above process, we obtain, for each document D , two distributions – (i) the model-assigned attention distribution $\{p_{D,i}^M\}$, and, (ii) the expert-assigned importance distribution $\{q_{D,i}^{M,\mathcal{E}}\}$, for every expert $\mathcal{E}$ and every model M . To measure the agreement of expert $\mathcal{E}$ with the model M , we compute the KL-Divergence score between these two distributions. Note that a lower value of the KL-divergence indicates a better match between the expert and the model.

5.5.3 Results. We average the KL-divergence scores across all $D \in$ ILDC-expert (set of 56 documents) to obtain a single score for a given expert $\mathcal{E}$ and a given model M . This process is repeated across all experts and all models and the KL-divergence values are stated in Table 6. For 4 out of 5 experts, we see that InLegalBERT has a lower KL-divergence score than LegalBERT, whereas InCaseLawBERT has a lower score than CaseLawBERT for all 5 experts. This corroborates with the fact that CJPE performance (macro-F1 over the test set of ILDC-multi, reported in Table 5) improves upon re-training on Indian data. Both InLegalBERT and InCaseLawBERT achieve lower scores than CustomInLawBERT for majority of the experts.

Interestingly, CaseLawBERT has comparatively poor scores of KL-divergence despite having a decent performance in terms of macro-F1. Also, InLegalBERT attains a better macro-F1 score than InCaseLawBERT (Table 5), but InCaseLawBERT shows greater agreement with the experts than InLegalBERT (Table 6). It is noteworthy that the macro-F1 has been calculated over the test set in the ILDC-multi dataset, whereas the KL-Divergence scores are calculated over ILDC-expert, which is a small subset of the former.

6 CONCLUSION AND FUTURE WORK

In this work, we have investigated the gains of re-training existing pre-trained models for the legal domain, namely, LegalBERT [9] and CaseLawBERT [33], on a large corpus of Indian legal data. We also pre-trained a BERT-base model from scratch on the same corpus. Based on the results across three practically important legal NLP tasks over standard datasets from India as well as from other domains/countries – Legal Statute Identification, Semantic Segmentation, and Court Judgement Prediction – we can conclude that domain-specific pre-training on Indian legal text helps, in general, to improve the performances for both the above models.

We observe some common trends about various PLMs in the legal domain: (i) When re-trained on Indian data, InLegalBERT improves significantly over LegalBERT, while the gains are much smaller for InCaseLawBERT over CaseLawBERT for Indian datasets. In fact, InLegalBERT attains the best F1 scores across all the three tasks and five datasets, which shows that the LegalBERT model

is more adaptable to re-training on new data, as compared to the CaseLawBERT model, while also improving in performance on old (EU) data. (ii) CustomInLawBERT comes close in terms of macro-F1 to both InLegalBERT and InCaseLawBERT across multiple datasets despite having being trained for much lesser number of steps (700k vs. 1.3M vs. 2.3M steps respectively). (iii) For 4 out of the 5 datasets we have experimented with, InLegalBERT (available at <https://huggingface.co/law-ai/InLegalBERT>) outperforms the result in the original papers (which introduced the respective dataset).

In general, these experiments show that there is promise in developing country-specific legal PLMs, especially if there are linguistic idiosyncrasies (as seen in Indian legal documents). So, what should be the prospective way of developing such a PLM – (i) Re-training an existing legal PLM on the domain-specific data, or (ii) Pre-train an LM from scratch, preferably using domain-specific vocabulary? Although prior works have shown that the latter is usually a better approach [9, 33], the choice might actually be dependent on factors such as size of pre-training data, or the availability of computing resources. In case the number of pre-training steps is limited, re-training an existing legal PLM might be a promising approach, to obtain improvements over the vanilla variants. Since training from scratch naturally requires a lot of pre-training steps, this approach can underperform in the case of a limited pre-training budget. Moreover, re-trained PLMs have the advantage of having being trained on legal data from multiple countries.

The PLMs InLegalBERT, InCaseLawBERT and CustomInLawBERT, as well as the code for pre-training and fine-tuning are available publicly to the research community (see Section 1).

In the future, we plan to further investigate with more sophisticated architectures for pre-training. In particular, hierarchical encoding methods can be useful for exploiting the structure of the text of legal documents. We also wish to conduct more experiments to investigate the pre-existing biases in the PLMs, and how these are affected by re-training on Indian data.

Acknowledgements: Shounak Paul is supported by the Prime Minister’s Research Fellowship (PMRF) from the Government of India. We acknowledge National Supercomputing Mission (NSM) for providing computing resources of ‘PARAM Shakti’ at IIT Kharagpur, which is implemented by C-DAC and supported by MeitY and DST, Government of India.

REFERENCES

- [1] Iz Beltagy, Kyle Lo, and Arman Cohan. 2019. SciBERT: A Pretrained Language Model for Scientific Text. In *Proc. of EMNLP-IJCNLP*.
- [2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150* (2020).
- [3] Paheli Bhattacharya, Kaustubh Hiware, Subham Rajgaria, Nilay Pochhi, Kripabandhu Ghosh, and Saptarshi Ghosh. 2019. A Comparative Study of Summarization Algorithms Applied to Legal Case Judgments. In *Proc. ECIR*.
- [4] Paheli Bhattacharya, Shounak Paul, Kripabandhu Ghosh, Saptarshi Ghosh, and Adam Wyner. 2019. Identification of Rhetorical Roles of Sentences in Indian Legal Judgments. In *Proc. of JURIX*.
- [5] Paheli Bhattacharya, Shounak Paul, Kripabandhu Ghosh, Saptarshi Ghosh, and Adam Wyner. 2021. DeepRhole: deep learning for rhetorical role labeling of sentences in legal case documents. *Artificial Intelligence and Law* (11 2021), 1–38.
- [6] Ilias Chalkidis et al. 2022. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. In *Proc. of ACL*.
- [7] Ilias Chalkidis, Ion Androutsopoulos, and Nikolaos Aletras. 2019. Neural Legal Judgment Prediction in English. In *Proc. of ACL*.
- [8] Ilias Chalkidis, Emmanouil Fergadiotis, Prodromos Malakasiotis, and Ion Androutsopoulos. 2019. Large-Scale Multi-Label Text Classification on EU Legislation. In *Proc. of ACL*.
- [9] Ilias Chalkidis, Manos Fergadiotis, Prodromos Malakasiotis, Nikolaos Aletras, and Ion Androutsopoulos. 2020. LEGAL-BERT: The Muppets straight out of Law School. In *Proc. of EMNLP*.
- [10] Jacob Devlin et al. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proc. of NAACL*.
- [11] Rotem Dror et al. 2018. The Hitchhiker’s Guide to Testing Statistical Significance in Natural Language Processing. In *Proc. of ACL*. 1383–1392.
- [12] Peter Henderson et al. 2022. Pile of Law: Learning Responsible Data Filtering from the Law and a 256GB Open-Source Legal Dataset. (2022).
- [13] Prathamesh Kalamkar, D. Janani Venugopalan Ph., and Vivek Raghavan Ph. D. 2021. Indian Legal NLP Benchmarks : A Survey. *arXiv e-prints* (2021).
- [14] Daniel Katz, II Bommarito, and Josh Blackman. 2016. A General Approach for Predicting the Behavior of the Supreme Court of the United States. *PLOS ONE* 12 (2016).
- [15] John D. Lafferty, Andrew McCallum, and Fernando Pereira. 2001. Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data. In *ICML*.
- [16] Jinhyuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, and Jaewoo Kang. 2019. BioBERT: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics* (2019).
- [17] Quentin Lhoest et al. 2021. Datasets: A Community Library for Natural Language Processing. In *Proc. of EMNLP: System Demonstrations*. 175–184.
- [18] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR abs/1907.11692* (2019).
- [19] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *ICLR*.
- [20] Vijit Malik, Rishabh Sanjay, Shubham Kumar Nigam, Kripabandhu Ghosh, Shouvik Kumar Guha, Arnab Bhattacharya, and Ashutosh Modi. 2021. Indian Legal Documents Corpus for Court Judgment Prediction and Explanation. In *Proc. ACL-IJCNLP*.
- [21] Masha Medvedeva, Michel Vols, and Martijn Wieling. 2020. Using machine learning to predict decisions of the European Court of Human Rights. *Artificial Intelligence and Law* (2020), 237–266.
- [22] Mahdi Naser Moghadasi and Yu Zhuang. 2020. Sent2Vec: A New Sentence Embedding Representation With Sentimental Semantic. In *Proc. Big Data*.
- [23] Adam Paszke et al. 2017. Automatic differentiation in PyTorch. In *NIPS-W*.
- [24] Shounak Paul, Pawan Goyal, and Saptarshi Ghosh. 2022. LeSICin: A Heterogeneous Graph-Based Approach for Automatic Legal Statute Identification from Indian Legal Documents. In *Proc. of AAAI*.
- [25] Pranav Rajpurkar et al. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. *CoRR abs/1606.05250* (2016).
- [26] Wilson L. Taylor. 1953. “Cloze Procedure”: A New Tool for Measuring Readability. *Journalism & Mass Communication Quarterly* 30 (1953), 415 – 433.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Proc. NeurIPS*.
- [28] Alex Wang et al. 2018. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. In *Proc. of 2018 EMNLP Workshop BlackboxNLP*.
- [29] Thomas Wolf et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proc. of EMNLP: System Demonstrations*. 38–45.
- [30] Yonghui Wu et al. 2016. Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. *CoRR abs/1609.08144* (2016).
- [31] Chaojun Xiao, Xueyu Hu, Zhiyuan Liu, Cunchao Tu, and Maosong Sun. 2021. Lawformer: A pre-trained language model for Chinese legal long documents. *AI Open* 2 (2021), 79–84.
- [32] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime G. Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. In *NeurIPS*.
- [33] Lucia Zheng, Neel Guha, Brandon R. Anderson, Peter Henderson, and Daniel E. Ho. 2021. Assessing Self-Supervised Learning for Law and the CaseHOLD Dataset of 53,000+ Legal Holdings. In *Proc. of ICAIL*.
- [34] Haoxi Zhong, Chaojun Xiao, Cunchao Tu, Tianyang Zhang, Zhiyuan Liu, and Maosong Sun. 2020. How Does NLP Benefit Legal System: A Summary of Legal Artificial Intelligence. In *Proc. ACL*.